

SCS 1309

Database Management Systems

Lecture 10 – Relational Calculus

MVPT Lakshika,
Lecturer (Probationary),
Department of Information Systems Engineering, UCSC.
Contact: tlv@ucsc.cmb.ac.lk

Outline

Relational Calculus

- Tuple Relational Calculus (TRC)
- Domain Relational Calculus (DRC)

Relational Query Languages

Two mathematical Query Languages form the basis for real query languages (e.g. SQL), and for implementation:

- **Relational Algebra:** More operational, very useful for representing execution plans. Basis for SEQUEL.
- **Relational Calculus:** Describe WHAT they want, rather than HOW to compute/ retrieve it
 - Basis for QUEL.
 - A non-procedural query language: Declarative (unlike Relational Algebra)
 - Based on logic & predicates (i.e., conditions).

Introduction to Relational Calculus

- A **certain order** is always explicitly specified in a **relational algebra expression**, and a strategy for evaluating the query is implied.
- In the relational calculus, there is **no description** of **how to evaluate a query**.
 - A relational calculus query specifies **what** is to be retrieved rather than **how** to retrieve it.
- The relational calculus is **not related to** differential and integral calculus in Math.
 - But takes its name from a branch of symbolic logic called **predicate calculus**.
 - In predicate calculus, a **predicate** is a truth-valued function with arguments.
 - When we substitute values for the arguments, the function yields an expression, called a **proposition**, which can be either true or false.
- Two forms: **Tuple Relational Calculus (TRC)** and **Domain Relational Calculus (DRC)**

Introduction to Relational Calculus

Example: 'John White is a member of staff' and 'John White earns more than Ann Beech' are both propositions (Because we can determine whether they are true or false)

Sentence 1: We have a function, 'is a member of staff', with one argument (John White).

Sentence 2: We have a function, 'earns more than', with two arguments (John White and Ann Beech).

- If a predicate contains a variable, as in ' x is a member of staff', there must be an associated **range** for x .
 - The proposition may be **true** for some values of this range for x , but for other values, it may be **false**.
- If P is a predicate, then we can write the set of all x such that P is true for x , as:
 $\{ x \mid P(x) \}$, The set of all x such that P of x is true.

Tuple Relational Calculus (TRC)

Introduction to TRC

- Find tuples for which a predicate is true.
- The calculus is based on the use of **tuple variables**.
 - A **tuple variable** is a variable that **ranges over** a named relation (a variable whose only permitted values are **tuples of the relation**).
- General Form: $\{t \mid P(t)\}$ where **t** is the tuple variable and **P(t)** is the predicate (condition that the tuple must satisfy)

Example: to specify the range of a tuple variable **S** as the **Staff** relation, we write:

Staff(S)

- To express the query 'Find the set of all tuples **S** such that **F(S)** is true', we can write:

{S | F(S)}, where **F** is called a formula.

TRC

Logical operators: The logical connectives $\wedge$ (**AND**), $\vee$ (**OR**), and $\neg$ (**NOT**) are used to form compound predicates.

Example 1: Express the query 'Find the staffNo, fName, lName, position, sex, DOB, salary, and branchNo of all staff earning more than 10,000'.

$\{S \mid \text{Staff}(S) \wedge S.\text{salary} > 10000\} \text{ or } \{S \mid \text{Staff}(S) \wedge S[\text{salary}] > 10000\}$

Meaning: Find S such that S is in Staff AND S has a salary over 10000.

Here, **S** represents tuples from *Staff*, and **S.salary** or **S[salary]** means the value of the *salary* attribute for the tuple variable **S**.

Example 2: To retrieve a particular attribute, such as *salary*, we would write:

$\{S.\text{salary} \mid \text{Staff}(S) \wedge S.\text{salary} > 10000\}$

TRC

Activity 1: Retrieve only the first and last names of all staff earning more than 50,000.

$\{S.fName, S.lName \mid Staff(S) \text{ AND } S.Salary > 50000\}$

Or

$\{S.fName, S.lName \mid Staff(S) \wedge S.Salary > 50000\}$

Or

$\{S[fName], S[lName] \mid Staff(S) \wedge S[Salary] > 50000\}$

TRC

- TRC uses **logical operators**:
 - $\wedge$ (**AND**) – Both conditions must be true
 - $\vee$ (**OR**) – At least one condition must be true
 - $\neg$ (**NOT**) – Negates a condition
- TRC uses two **quantifiers** with a formula to tell how many instances the predicate applies to.
 - $\exists$, Existential (**Exists**) – At least one tuple satisfies the condition
 - $\forall$, Universal (**For all**) – All tuples satisfy the condition

The Existential and Universal quantifiers

- The **existential quantifier** $\exists$ ('there exists') is used in a formula that **must be true for at least one instance**.

$\exists \mathbf{t} \in \mathbf{R} (\text{condition})$ – This means there exists at least one tuple $\mathbf{t}$ in relation $\mathbf{R}$ that satisfies the condition.

Example: $\text{Staff}(S) \wedge (\exists B) (\text{Branch}(B) \wedge (B.\text{branchNo} = S.\text{branchNo}) \wedge B.\text{city} = \text{'London'})$

This means, 'There exists a *Branch* tuple that has the same *branchNo* as the *branchNo* of the current *Staff* tuple, S , and is located in London'.

- The **universal quantifier** $\forall$ ('for all') is used in statements about **every instance**.

$\forall \mathbf{t} \in \mathbf{R} (\text{condition})$ – This means for every tuple $\mathbf{t}$ in relation $\mathbf{R}$, the condition must be true.

Example: $(\forall B) (\text{Branch}(B) \rightarrow B.\text{city} \neq \text{'Paris'})$

This means, 'For all *Branch* tuples, the address is not in Paris'.

The existential and universal quantifiers

- **Tuple variables** that are qualified by $\forall$ or $\exists$ are called **bound variables**, otherwise the tuple variables are called **free variables**.
- The only **free variables** in a relational calculus expression should be those on the left side of the bar (**|**).

Example: In the following query:

$\{S.fName, S.lName \mid Staff(S) \wedge (\exists B) (Branch(B) \wedge (B.branchNo = S.branchNo) \wedge B.city = 'London')\}$

Free Variable: **S** (Range: Staff), **S** is bound successively to each tuple of *Staff*. Its attributes *fName* and *lName* are what the query actually returns.

Bound Variable: **B** (Range: Branch)

TRC – More Examples

1. List the names of all managers who earn more than 25,000.

$\{S.fName, S.lName \mid Staff(S) \wedge S.position = 'Manager' \wedge S.salary > 25000\}$

2. List the staff who manage properties for rent in Glasgow.

$\{S \mid Staff(S) \wedge (\exists P) (PropertyForRent(P) \wedge (P.staffNo = S.staffNo) \wedge P.city = 'Glasgow')\}$

- You could reformulate the query as: 'For each member of staff whose details we want to list, there exists a tuple in the relation *PropertyForRent* for that member of staff with the value of the attribute city in that tuple being Glasgow.'

TRC – More Examples

3. List the names of staff who currently do not manage any properties.

$$\{S.fName, S.lName \mid Staff(S) \wedge (\sim(\exists P) (PropertyForRent(P) \wedge (S.staffNo = P.staffNo)))\}$$

or

$$\{S.fName, S.lName \mid Staff(S) \wedge ((\forall P) (\sim PropertyForRent(P) \vee \sim(S.staffNo = P.staffNo)))\}$$

4. List the names of clients who have viewed a property for rent in Glasgow.

$$\{C.fName, C.lName \mid Client(C) \wedge ((\exists V) (\exists P) (Viewing(V) \wedge PropertyForRent(P) \wedge (C.clientNo = V.clientNo) \wedge (V.propertyNo = P.propertyNo) \wedge P.city = 'Glasgow'))\}$$

TRC – More Examples

5. List all cities where there is either a branch office or a property for rent.

Relational algebra expression: $\Pi_{\text{city}}(\text{Branch}) \cup \Pi_{\text{city}}(\text{PropertyForRent})$

$\{\text{T.city} \mid (\exists B) (\text{Branch}(B) \wedge B.\text{city} = \text{T.city}) \vee (\exists P) (\text{PropertyForRent}(P) \wedge P.\text{city} = \text{T.city})\}$

* You need a neutral variable (**T**) that can represent a city regardless of which table it was found in.

6. List all the cities where there is a branch office but no properties for rent.

Relational algebra expression: $\Pi_{\text{city}}(\text{Branch}) - \Pi_{\text{city}}(\text{PropertyForRent})$

$\{B.\text{city} \mid \text{Branch}(B) \wedge (\sim(\exists P) (\text{PropertyForRent}(P) \wedge B.\text{city} = P.\text{city}))\}$

TRC – More Examples

7. List all the cities where there is both a branch office and at least one property for rent.

Relational algebra expression: $\Pi_{\text{city}}(\text{Branch}) \cap \Pi_{\text{city}}(\text{PropertyForRent})$

$\{B.\text{city} \mid \text{Branch}(B) \wedge ((\exists P) (\text{PropertyForRent}(P) \wedge B.\text{city} = P.\text{city}))\}$

Domain Relational Calculus (DRC)

Introduction to DRC

- TRC uses variables that range over tuples in a relation.
- DRC also uses variables, but variables take their individual values from domains of attributes rather than tuples of relations.
- General form: $\{d_1, d_2, \dots, d_n \mid F(d_1, d_2, \dots, d_m)\} \quad m \geq n$

where $d_1, d_2, \dots, d_n, \dots, d_m$ represent domain variables and $F(d_1, d_2, \dots, d_m)$ represents a formula composed of atoms (predicate condition), where each atom has one of the following forms:

- $R(d_1, d_2, \dots, d_n)$, where R is a relation of degree n and each d_i is a domain variable.
- $d_i \theta d_j$, where d_i and d_j are domain variables, and θ is one of the comparison operators ($<, \leq, >, \geq, =, \neq$); the domains d_i and d_j must have members that can be compared by θ .
- $d_i \theta c$, where d_i is a domain variable, c is a constant from the domain of d_i , and θ is one of the comparison operators.

DRC

- You can recursively build up a formula from **atoms** using the following rules:
 - If F_1 and F_2 are formula, so are their conjunction $F_1 \wedge F_2$, their disjunction $F_1 \vee F_2$, and the negation $\sim F_1$.
 - If F is a formula with domain variable X , then $(\exists X)(F)$ and $(\forall X)(F)$ are also formula.

Example 1: Find employees who earn more than 50,000.

Relation: Employee (EmployeeID, Name, Salary, DepartmentID)

DRC Query: $\{E, N, S, D \mid \text{Employee}(E, N, S, D) \wedge S > 50000\}$

Explanation: $E \rightarrow$ Represents the Employee ID, $N \rightarrow$ Represents the Name, $S \rightarrow$ Represents the Salary, $D \rightarrow$ Represents the Department ID. $S > 50000$ This ensures that only employees with salaries greater than 50,000 are selected.

DRC

Example 2: Find the names of all managers who earn more than £25,000.

$\{fN, IN \mid (\exists sN, posn, sex, DOB, sal, bN) (Staff(sN, fN, IN, posn, sex, DOB, sal, bN) \wedge posn = \text{'Manager'} \wedge sal > 25000))\}$

The equivalent TRC: $\{S.fName, S.lName \mid Staff(S) \wedge S.position = \text{'Manager'} \wedge S.salary > 25000\} \rightarrow$ Here, each attribute is given a (variable) name.

- The condition $Staff(sN, fN, \dots, bN)$ ensures that the domain variables are restricted to be attributes of the same tuple. Thus, you can use the formula $posn = \text{'Manager'}$, rather than $Staff.position = \text{'Manager'}$.
- In the TRC, when we write $\exists posn$ for some tuple variable $posn$, we bind the variable to the relation $Staff$ by writing $Staff(posn)$.

DRC

Example 2: Find the names of all managers who earn more than £25,000.

$\{fN, IN \mid (\exists sN, posn, sex, DOB, sal, bN) (Staff(sN, fN, IN, posn, sex, DOB, sal, bN) \wedge posn = \text{'Manager'} \wedge sal > 25000))\}$

The equivalent TRC: $\{S.fName, S.lName \mid Staff(S) \wedge S.position = \text{'Manager'} \wedge S.salary > 25000\} \rightarrow$ Here, each attribute is given a (variable) name.

- On the other hand, in the DRC, *posn* refers to a domain value and remains unconstrained until it appears in the subformula *Staff(sN, fN, IN, posn, sex, DOB, sal, bN)* when it becomes constrained to the position values that appear in the *Staff* relation.

DRC – More Examples

1. List the staff who manage properties for rent in Glasgow.

$$\{sN, fN, lN, posn, sex, DOB, sal, bN \mid (\exists sN1, cty) (Staff(sN, fN, lN, posn, sex, DOB, sal, bN) \wedge PropertyForRent(pN, st, cty, pc, typ, rms, rnt, oN, sN1, bN1) \wedge (sN = sN1) \wedge cty = 'Glasgow')\}$$

- This query can also be written as:

$$\{sN, fN, lN, posn, sex, DOB, sal, bN \mid (Staff(sN, fN, lN, posn, sex, DOB, sal, bN) \wedge PropertyForRent(pN, st, 'Glasgow', pc, typ, rms, rnt, oN, sN, bN1))\}$$

- The domain variable *cty* in *PropertyForRent* has been replaced with the constant 'Glasgow'.
- The same domain variable *sN*, which represents the staff number, has been repeated for *Staff* and *PropertyForRent*.

DRC – More Examples

2. List the names of staff who currently do not manage any properties for rent.

$$\{fN, IN \mid (\exists sN) (Staff(sN, fN, IN, posn, sex, DOB, sal, bN) \wedge (\sim(\exists sN1) (PropertyForRent(pN, st, cty, pc, typ, rms, rnt, oN, sN1, bN1) \wedge (sN = sN1)))) \}$$

3. List the names of clients who have viewed a property for rent in Glasgow.

$$\{fN, IN \mid (\exists cN, cN1, pN, pN1, cty) (Client(cN, fN, IN, tel, pT, mR) \wedge Viewing(cN1, pN1, dt, cmt) \wedge PropertyForRent(pN, st, cty, pc, typ, rms, rnt, oN, sN, bN) \wedge (cN = cN1) \wedge (pN = pN1) \wedge cty = 'Glasgow')) \}$$

4. List all cities where there is either a branch office or a property for rent.

$$\{cty \mid (Branch(bN, st, cty, pc) \vee PropertyForRent(pN, st1, cty, pc1, typ, rms, rnt, oN, sN, bN1)) \}$$

DRC – More Examples

5. List all the cities where there is a branch office but no properties for rent.

$$\{\text{cty} \mid (\text{Branch}(\text{bN}, \text{st}, \text{cty}, \text{pc}) \wedge (\sim(\exists \text{cty1}) (\text{PropertyForRent}(\text{pN}, \text{st1}, \text{cty1}, \text{pc1}, \text{typ}, \text{rms}, \text{rnt}, \text{oN}, \text{sN}, \text{bN1}) \wedge (\text{cty} = \text{cty1}))))))\}$$

6. List all the cities where there is both a branch office and at least one property for rent.

$$\{\text{cty} \mid (\text{Branch}(\text{bN}, \text{st}, \text{cty}, \text{pc}) \wedge (\exists \text{cty1}) (\text{PropertyForRent}(\text{pN}, \text{st1}, \text{cty1}, \text{pc1}, \text{typ}, \text{rms}, \text{rnt}, \text{oN}, \text{sN}, \text{bN1}) \wedge (\text{cty} = \text{cty1}))))\}$$

TRC VS DRC

Feature	TRC (Tuple Relational Calculus)	DRC (Domain Relational Calculus)
Representation	Uses tuple variables	Uses domain variables
Focus	Works with entire tuples (horizontal row)	Works with specific values (vertical specific values)
Example Query	$\{ t \mid t \in \text{Employee} \wedge t.\text{Salary} > 50000 \}$	$\{ \langle E, N, S, D \rangle \mid \text{Employee}(E, N, S, D) \wedge S > 50000 \}$
Explanation of Query	Retrieves all tuples t from Employee where Salary > 50000	Retrieves specific domain values (E = EmpID, N = Name, S = Salary, D = DeptID) where Salary > 50000
More Readable	Easier	Not easier
Use Cases	Used in relational databases (like SQL)	Theoretical & less commonly implemented

Any Questions?